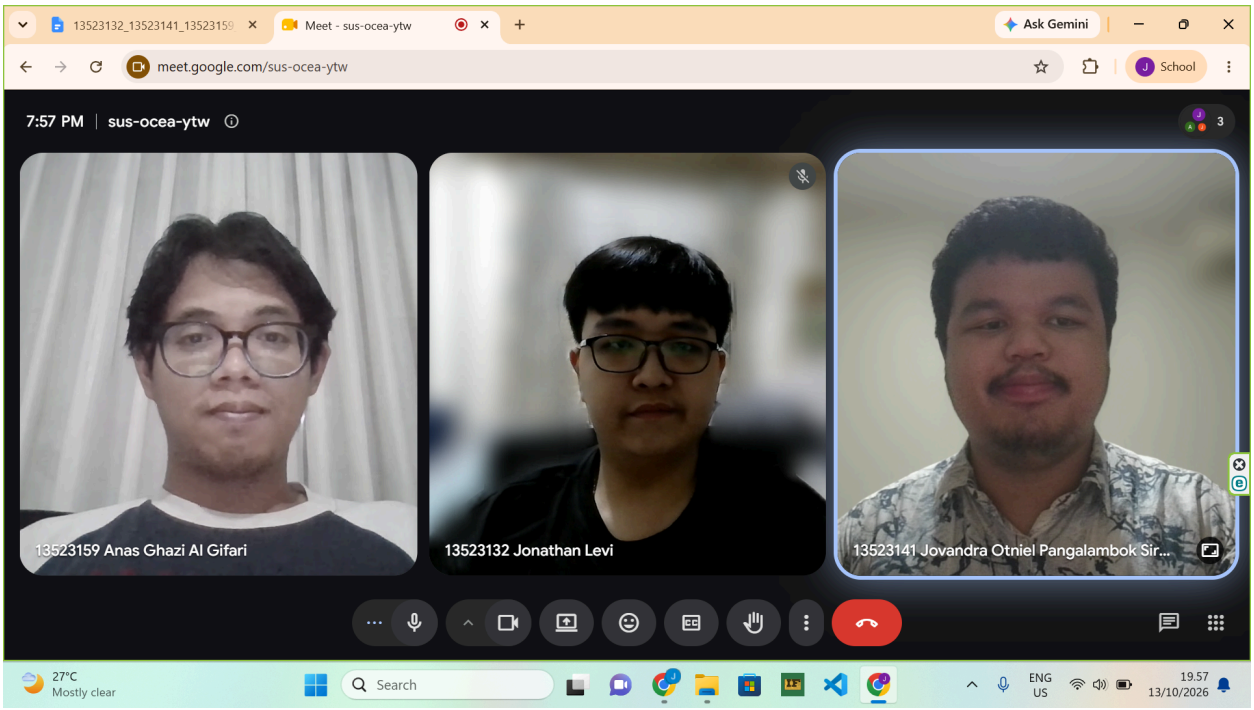
Tugas 2
IF4020 Kriptografi

Membuat Block Cipher “Baru”



Disusun oleh:

Jonathan Levi 13523132

Jovandra Otniel P. S. 13523141

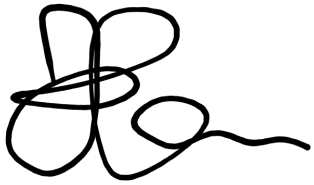
Anas Ghazi Al Gifari 13523159

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG

2026

PERNYATAAN TIDAK MELAKUKAN KECURANGAN

Tugas dan laporan ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.







Jonathan Levi	Jovandra Otniel P. S.	Anas Ghazi Al Gifari

BAB I
PENDAHULUAN

1.1 Latar Belakang

Block cipher merupakan algoritma enkripsi simetris yang mengamankan data dengan membagi pesan ke dalam blok-blok bit berukuran tetap dan mengolahnya melalui beberapa putaran transformasi. Untuk menjamin keamanannya, rancangan algoritma harus memenuhi prinsip *confusion* dan *diffusion* menggunakan operasi substitusi serta transposisi. Pada tugas ini, dilakukan perancangan dan implementasi algoritma *block cipher* baru beserta pengujian performa keamanannya yang meliputi *avalanche effect*, entropi, dan distribusi histogram dalam berbagai mode operasi enkripsi.

1.2 *Review* Singkat Block Cipher Sejenis

- DES:
  Menggunakan Feistel Network (16 putaran) dengan ukuran blok 64 bit dan ukuran key 64 bit (56 + 8 bit parity). Mekanisme putarannya, yaitu blok dibagi menjadi L (32 bit) dan R (32 bit). Fungsi F(R, K) menggunakan Expansion Box (32 -> 48 bit), XOR dengan kunci putaran (48 bit), substitusi melalui 8 buah S-box 6-to-4 bit yang berbeda, dan permutasi bit P (32 bit).
  Perbedaanya dengan algoritma kami, block cipher ini menyederhanakan fungsi F tanpa expansion box (R dan K sama-sama 32 bit), menggunakan satu jenis S-box 4-to-4 bit secara seragam, serta menambahkan circular shift 3 bit setelah permutasi.

1.3 Pendekatan Utama

Cipher blok (*block cipher*) yang akan dirancang pada tugas kali ini mencakup beberapa konsep yang lazim dalam kriptografi kunci simetris, yaitu tabel substitusi (s-box), tabel permutasi (p-box), jaringan feistel, dan *padding* PKCS#7. Substitusi berfungsi untuk mengaburkan hubungan statistik antara kunci dan *ciphertext*. Hal ini meningkatkan *confusion* dalam algoritma cipher blok ini. Permutasi berfungsi untuk memudarkan hubungan statistik (frekuensi) antara *plaintext* dan *ciphertext*. Hal ini meningkatkan keamanan cipher blok dalam hal *diffusion*.

BAB II
DASAR TEORI

2.1 Cipher Blok

Cipher blok (*block cipher*) adalah algoritma kriptografi kunci simetri yang membagi plainteks menjadi beberapa blok. Setiap blok kemudian dienkripsi menggunakan fungsi enkripsi yang terdiri dari tahapan permutasi dan substitusi. Tidak seperti cipher alir yang mengenkripsi plainteks bit demi bit, cipher blok mengelompokkan satu plainteks besar menjadi beberapa bagian plainteks kecil yang disebut blok. Hal ini melahirkan beberapa mode cipher blok yang terkenal, seperti *electronic codebook*, *cipher block chaining*, *cipher feedback*, *output feedback*, dan *counter*.

2.2 Prinsip *Confusion* dan *Diffusion*

Keamanan algoritma kriptografi haruslah diuji sedemikian sehingga cipherteks yang dihasilkan oleh algoritma tersebut sulit dipecahkan oleh para kriptanalis. Menurut Claude Shannon, keamanan algoritma kriptografi dapat ditinjau dari 2 sifat, yaitu *confusion* (kebingungan) dan *diffusion* (penyebaran bagaikan difusi). Dua sifat ini menjadi parameter bagi praktisi kriptografer untuk mengukur "seberapa aman" algoritma kriptografi.

Confusion adalah tahapan algoritma kriptografi yang diterapkan sedemikian sehingga penyerang kesulitan mengenali korelasi antara kunci (*key*) dengan cipherteks. Prinsip ini diimplementasikan melalui operasi substitusi. Hal ini bertujuan untuk mempersulit kriptanalis dalam menebak kunci dari cipherteks.

Diffusion adalah tahapan algoritma kriptografi yang diterapkan sedemikian sehingga penyerang kesulitan mencari hubungan statistik antara plainteks dan cipherteks melalui analisis frekuensi. Hal ini berarti bit atau huruf pada plainteks ditransformasikan menjadi bit atau huruf cipherteks yang berbeda meskipun bit atau huruf asalnya (plainteks) sama. Pada algoritma kriptografi, prinsip ini diimplementasikan melalui operasi permutasi.

2.3 Jaringan Feistel

Jaringan Feistel merupakan struktur desain yang menggambarkan alur enkripsi cipher blok. Pada setiap blok, terdapat perpaduan algoritma kriptografi yang melibatkan substitusi, permutasi, dan pembangkitan kunci ronde. Setiap blok pada jaringan Feistel dapat didesain sedemikian sehingga hubungan antar blok fleksibel. Hubungan antar blok dapat berupa hubungan saling lepas (ECB, CTR) atau hubungan saling mengikat (CBC, CFB, OFB). Pada tugas ini, jaringan Feistel mengikuti standar Feistel, yaitu pemecahan blok (64 bit) menjadi 2 blok 32-bit, dimana blok kiri dan kanan mengalami iterasi sebanyak jumlah ronde. Pada setiap iterasi, posisi blok kiri dan

kanan ditukar. Blok kanan lama menjadi blok kiri baru, sedangkan blok kiri lama di-XOR dengan fungsi ronde(blok kanan lama, kunci ronde) kemudian hasilnya menjadi blok kanan baru.

BAB III
PERANCANGAN CIPHER BLOK

3.1 Pembangkitan Kunci Ronde, Enkripsi, dan Dekripsi Cipher Blok

```python
def generate_round_keys(master_key):
    if len(master_key) != KEY_SIZE:
        raise ValueError(f"Master key harus berukuran {KEY_SIZE} bytes.")
    w0, w1 = int.from_bytes(master_key[:4], 'big'), int.from_bytes(master_key[4:], 'big')
    w = [w0, w1]
    round_keys = []
    for i in range(ROUND_COUNT):
        round_key = (w[i % 2] + ((i + 1) * GOLDEN_RATIO)) & 0xFFFFFFFF
        round_key = circular_shift_32(round_key, (i * 3) % 32)
        round_keys.append(round_key)
    return round_keys
```

Fungsi Pembangkitan Kunci Ronde

Pada algoritma ini, digunakan fungsi pembangkitan ronde yang membangkitkan kunci ronde dari kunci utama (*master key* 8 karakter). *Master key* dibagi menjadi 2 bagian, *w0* dan *w1*. Kemudian, terdapat perulangan dari 0 hingga ROUND_COUNT-1 dimana round_key didapat dari penjumlahan antara "*w0* atau *w1* (tergantung indeks iterasinya)" dengan "perkalian dari indeks + 1 dan rasio emas". Kemudian, round_key dirotasi ke kiri sebanyak "indeks kali 3 mod 32" dan ditambahkan ke *array* round_keys. Senarai round_keys menyimpan beberapa round_key sebanyak jumlah ronde yang telah dispesifikasikan di konstanta ROUND_COUNT sebelumnya.

```python
def encrypt_block(block_64bit, round_keys):
    l_i = block_64bit >> 32
    r_i = block_64bit & 0xFFFFFFFF # singkatan 32 angka "1" (0b111...111) dlm binary

    for i in range(ROUND_COUNT):
        key = round_keys[i]

        # Jaringan Feistel
        l_i_plus_1 = r_i
            # R_i+1 = L_i XOR f(R_i, K_i)
        r_i_plus_1 = l_i ^ f_function(r_i, key)

        l_i = l_i_plus_1
        r_i = r_i_plus_1

    result = (r_i << 32) | l_i
    return result
```

Fungsi Enkripsi Blok

Pada fungsi *encrypt_block*, diterapkan desain jaringan Feistel dimana pada setiap blok dilakukan pemisahan blok 64 bit menjadi 2 blok 32 bit yang terdiri dari blok kiri dan blok kanan. Kemudian, terdapat perulangan sebanyak ROUND_COUNT (pada aplikasi ini, ditetapkan 16 kali dari indeks 0 ke 15). Pada setiap iterasi, dijalankan operasi XOR antara operandi blok kiri (l_i) dan fungsi *f_function* dengan argumen fungsi r_i dan *key* (kunci_ronde). Hasil operasi tersebut disimpan pada r_i_plus_1. Setelah itu, posisi r_i diubah menjadi l_i_plus_1. Setelah itu, l_i_plus_1 menjadi l_1 dan r_i_plus_1 menjadi r_1. Setelah itu, l_1 digeser ke kiri sebanyak 32 bit dan digabung dengan r_1. Fungsi ini mengembalikan blok yang telah dienkripsi.

Bagian `l_i_plus_1 = r_i` dan `r_i_plus_1 = l_i ^ f_function(r_i, key)` memenuhi prinsip diffusion.

```python
def decrypt_block(block_64bit, round_keys):
    r_i = block_64bit >> 32
    l_i = block_64bit & 0xFFFFFFFF # singkatan 32 angka "1" (0b111...111) dlm binary

    for i in range(ROUND_COUNT - 1, -1, -1):
        key = round_keys[i]

        # Jaringan Feistel
        r_i_minus_1 = l_i
            # L_i+1 = R_i XOR f(L_i, K_i)
        l_i_minus_1 = r_i ^ f_function(l_i, key)

        r_i = r_i_minus_1
        l_i = l_i_minus_1

    result = (l_i << 32) | r_i
    return result
```

Fungsi Dekripsi Blok

Pada fungsi *decrypt_block*, diterapkan desain jaringan Feistel yang serupa dengan *encrypt_block* dimana pada setiap blok 64 bit dilakukan pemisahan blok menjadi 2 blok 32 bit yang terdiri dari blok kiri dan blok kanan. Kemudian, terdapat perulangan mundur (pada aplikasi ini, ditetapkan 16 kali, dari indeks 15 ke 0). Pada setiap iterasi perulangan, dijalankan operasi XOR antara operandi blok kanan (r_1) dan fungsi *f_function* dengan argumen fungsi l_1 dan *key* (kunci ronde). Hasil operasi tersebut disimpan pada l_i_minus_1. Setelah itu, posisi l_i diubah menjadi r_i_minus_1. Setelah itu, r_i_minus_1 menjadi r_1 dan l_i_minus_1 menjadi l_1. Setelah itu, l_1 digeser ke kiri sebanyak 32 bit dan digabung dengan r_1. Fungsi ini mengembalikan blok yang telah didekripsi.

3.2 Komponen Substitusi, Permutasi, dan Fungsi Putaran

```python
BLOCK_SIZE = 8 # 64 bits (8 bytes)
KEY_SIZE = 8 # 64 bits (8 bytes)
ROUND_COUNT = 16 # iterated cipher 16 rounds
GOLDEN_RATIO = 0x9E3779B9 # golden ratio for key schedule, 32 bits (4 bytes)

# S-Box 4 bit
SBOX = [3, 13, 10, 11, 8, 15, 4, 5,
        6, 9, 2, 7, 0, 1, 14, 12]

# P-Box 32 bit (feistel, masing-masing kiri dan kanan 32 bit)
PBOX = [25, 31, 20, 11, 9, 4, 18, 1,
        17, 6, 29, 16, 28, 15, 2, 0,
        23, 13, 12, 22, 19, 24, 30, 21,
        5, 14, 26, 27, 10, 7, 3, 8]
```

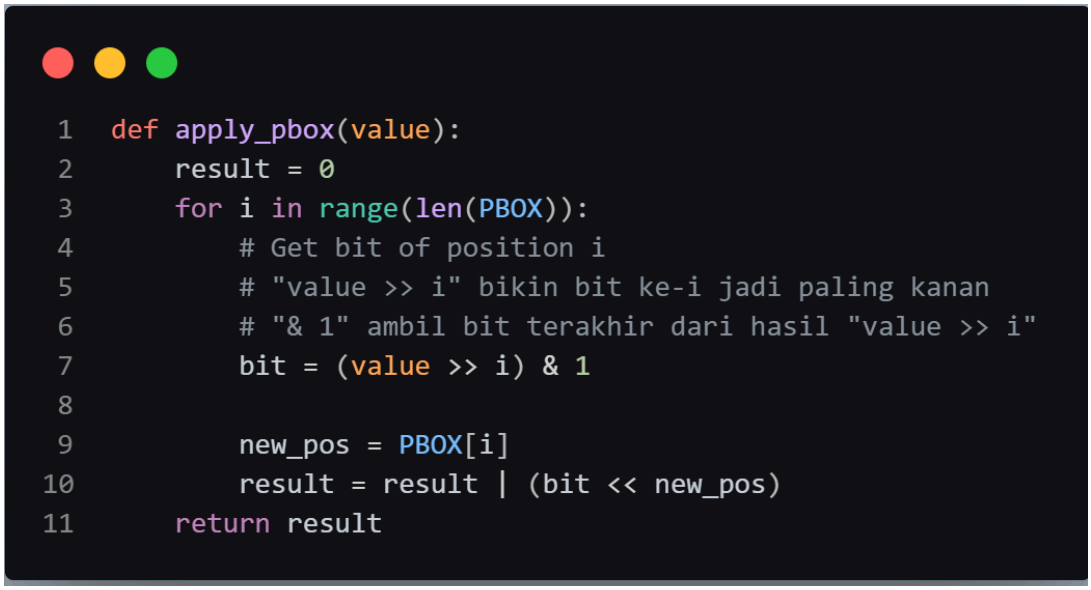
Konstanta ukuran blok, ukuran kunci, jumlah ronde, dan rasio emas

Bagian ini merupakan arsitektur yang dipilih, yaitu ukuran blok dan kunci 64 bit (8 byte), dengan Jaringan Feistel 16 putaran, dan nilai acak S-box dan P-box sedemikian rupa.

```python
def apply_sbox(value_32bit):
    result = 0
    for i in range(8):
        # Ambil per 4 bit, dari kanan
        chunk = (value_32bit >> (i * 4)) & 0b1111
        substitution = SBOX[chunk]
        result = result | (substitution << (i * 4))
    return result
```

Fungsi Substitusi

Bagian ini merupakan aplikasi S-box pada input 32 bit, dengan cara ambil per 4 bit dari kanan, lalu mengganti blok bit tersebut dengan nilai S-box pada indeks ke-blok bit tersebut, lalu menggabungkan hasilnya.



```python
def apply_pbox(value):
    result = 0
    for i in range(len(PBOX)):
        # Get bit of position i
        # "value >> i" bikin bit ke-i jadi paling kanan
        # "& 1" ambil bit terakhir dari hasil "value >> i"
        bit = (value >> i) & 1

        new_pos = PBOX[i]
        result = result | (bit << new_pos)
    return result
```

Fungsi Permutasi

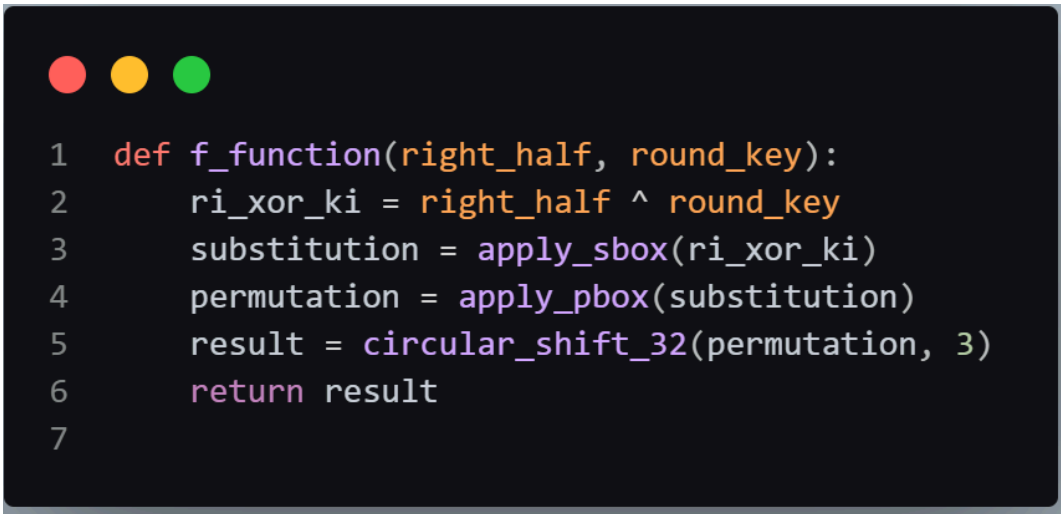
Bagian ini merupakan aplikasi P-box pada input yang mengambil sebuah bit (dimulai dari kanan), lalu menggeser bit tersebut sebanyak nilai P-box pada index ke-i, lalu menggabungkan hasilnya.



```python
def circular_shift_32(value, shift_amount):
    shift_amount %= 32
    shifted =  (value << shift_amount) | (value >> (32 - shift_amount))
    # Masking, karena "value << shift_amount" tidak membuang shift_amount bit terkiri
    mask = 0xFFFFFFFF # singkatan 32 angka "1" (0b111...111) dlm binary
    return shifted & mask
```

Fungsi Rotasi 32 bit

Bagian ini melakukan perputaran/rotasi bit, sehingga memindahkan bit terkiri ke bagian terkanan.



```python
def f_function(right_half, round_key):
    ri_xor_ki = right_half ^ round_key
    substitution = apply_sbox(ri_xor_ki)
    permutation = apply_pbox(substitution)
    result = circular_shift_32(permutation, 3)
    return result

```

Fungsi Ronde

Bagian ini mengimplementasikan fungsi f(...) pada Jaringan Feistel yang mengolah blok kanan bit dengan round key. `^ round_key` merupakan bagian yang memenuhi prinsip confusion, sedangkan `apply_pbox(substitution)`, `circular_shift_32(permutation, 3)` memenuhi prinsip diffusion.

3.3 Kelas NewBlockCipher Sebagai Enkapsulasi Algoritma Cipher Blok

```python
class NewBlockCipher:
    def __init__(self, master_key):
        if len(master_key) != KEY_SIZE:
            raise ValueError(f"Master key harus berukuran {KEY_SIZE} bytes.")
        self.round_keys = generate_round_keys(master_key)
        self.block_size = BLOCK_SIZE

    def encrypt_block_bytes(self, block_bytes):
        if len(block_bytes) != self.block_size:
            raise ValueError(f"Block harus berukuran {self.block_size} bytes.")
        block_int = int.from_bytes(block_bytes, 'big')
        encrypted_int = encrypt_block(block_int, self.round_keys)
        return encrypted_int.to_bytes(self.block_size, 'big')

    def decrypt_block_bytes(self, block_bytes):
        if len(block_bytes) != self.block_size:
            raise ValueError(f"Block harus berukuran {self.block_size} bytes.")
        block_int = int.from_bytes(block_bytes, 'big')
        decrypted_int = decrypt_block(block_int, self.round_keys)
        return decrypted_int.to_bytes(self.block_size, 'big')
```

Kelas ini membungkus metode enkripsi, metode dekripsi, atribut *round_keys*, dan atribut ukuran blok. Objek yang berasal dari instansiasi kelas tersebut mewakili algoritma NewBlockCipher yang siap digunakan oleh mode-mode cipher dalam menjalankan fungsi kriptografi. Selain itu, metode enkripsi dan dekripsi pada kelas NewBlockCipher juga berperan sebagai *adaptor* tipe data dari *bytes* ke integer sebelum fungsi utama kemudian dari integer ke *bytes* setelah fungsi utama selesai. Hal ini disebabkan fungsi utama enkripsi dan dekripsi blok hanya beroperasi jika diberikan input integer (bilangan bulat) dan hanya mengeluarkan nilai integer. Sehingga, metode enkripsi dan dekripsi pada kelas tersebut menjadi kompatibel dengan tipe data *bytes*.

3.4 Mode-Mode dalam Cipher Blok

```python
class CipherMode:
    def __init__(self, cipher : NewBlockCipher):
        self.cipher = cipher

    def encrypt(self, plaintext):
        raise NotImplementedError("Metode encrypt harus diimplementasikan oleh subclass.")

    def decrypt(self, ciphertext):
        raise NotImplementedError("Metode decrypt harus diimplementasikan oleh subclass.")
```

Kelas abstrak yang menjadi cetak biru kelas CipherMode. Semua mode cipher pada aplikasi ini merupakan turunan dari kelas ini.

3.4.1 Electronic Codebook (ECB)

```python
class ECBMode(CipherMode):
    def encrypt(self, plaintext):
        # Memastikan panjang plaintext adalah kelipatan dari ukuran blok
        padded_plaintext = PKCS7Padding.pad(plaintext, self.cipher.block_size)
        ciphertext = b''

        for i in range(0, len(padded_plaintext), self.cipher.block_size):
            block = padded_plaintext[i:i + self.cipher.block_size]
            encrypted_block = self.cipher.encrypt_block_bytes(block)
            ciphertext += encrypted_block

        return ciphertext

    def decrypt(self, ciphertext):
        if len(ciphertext) % self.cipher.block_size != 0:
            raise ValueError("Panjang ciphertext harus kelipatan dari ukuran blok.")

        decrypted_data = b''

        for i in range(0, len(ciphertext), self.cipher.block_size):
            block = ciphertext[i:i + self.cipher.block_size]
            decrypted_block = self.cipher.decrypt_block_bytes(block)
            decrypted_data += decrypted_block

        unpadded_data = PKCS7Padding.unpad(decrypted_data)
        return unpadded_data
```

Bagian ini mengimplementasikan mode Electronic Codebook dari cipher blok.

3.4.2 Cipher Block Chaining (CBC)

```python
class CBCMode(CipherMode):
    def __init__(self, cipher : NewBlockCipher, iv):
        super().__init__(cipher)
        if len(iv) != self.cipher.block_size:
            raise ValueError(f"IV harus berukuran {self.cipher.block_size} bytes.")
        self.iv = iv

    def encrypt(self, plaintext):
        padded_plaintext = PKCS7Padding.pad(plaintext, self.cipher.block_size)
        ciphertext = b''
        previous_block = self.iv

        for i in range(0, len(padded_plaintext), self.cipher.block_size):
            block = padded_plaintext[i:i + self.cipher.block_size]
            # XOR dengan blok sebelumnya (atau IV untuk blok pertama)
            block_to_encrypt = bytes(a ^ b for a, b in zip(block, previous_block))
            encrypted_block = self.cipher.encrypt_block_bytes(block_to_encrypt)
            ciphertext += encrypted_block
            previous_block = encrypted_block

        return ciphertext

    def decrypt(self, ciphertext):
        if len(ciphertext) % self.cipher.block_size != 0:
            raise ValueError("Panjang ciphertext harus kelipatan dari ukuran blok.")

        decrypted_data = b''
        previous_block = self.iv

        for i in range(0, len(ciphertext), self.cipher.block_size):
            block = ciphertext[i:i + self.cipher.block_size]
            decrypted_block = self.cipher.decrypt_block_bytes(block)
            # XOR dengan blok sebelumnya (IV untuk blok pertama)
            original_block = bytes(a ^ b for a, b in zip(decrypted_block, previous_block))
            decrypted_data += original_block
            previous_block = block

        unpadded_data = PKCS7Padding.unpad(decrypted_data)
        return unpadded_data
```

Bagian ini mengimplementasikan mode Cipher Block Chaining dari cipher blok.

3.4.3 Cipher Feedback (CFB)

```python
class CFBMode(CipherMode):
    def __init__(self, cipher : NewBlockCipher, iv):
        super().__init__(cipher)
        if len(iv) != self.cipher.block_size:
            raise ValueError(f"IV harus berukuran {self.cipher.block_size} bytes.")
        self.iv = iv

    def encrypt(self, plaintext):
        padded_plaintext = PKCS7Padding.pad(plaintext, self.cipher.block_size)
        ciphertext = b''
        feedback = self.iv

        for i in range(0, len(padded_plaintext), self.cipher.block_size):
            block = padded_plaintext[i:i + self.cipher.block_size]
            # Enkripsi feedback
            encrypted_feedback = self.cipher.encrypt_block_bytes(feedback)
            # XOR dengan blok plaintext
            encrypted_block = bytes(a ^ b for a, b in zip(block, encrypted_feedback))
            ciphertext += encrypted_block
            feedback = encrypted_block  # Update feedback dengan blok ciphertext yang baru

        return ciphertext

    def decrypt(self, ciphertext):
        if len(ciphertext) % self.cipher.block_size != 0:
            raise ValueError("Panjang ciphertext harus kelipatan dari ukuran blok.")

        decrypted_data = b''
        feedback = self.iv

        for i in range(0, len(ciphertext), self.cipher.block_size):
            block = ciphertext[i:i + self.cipher.block_size]
            # Enkripsi feedback
            encrypted_feedback = self.cipher.encrypt_block_bytes(feedback)
            # XOR dengan blok ciphertext
            original_block = bytes(a ^ b for a, b in zip(block, encrypted_feedback))
            decrypted_data += original_block
            feedback = block  # Update feedback dengan blok ciphertext yang baru

        unpadded_data = PKCS7Padding.unpad(decrypted_data)
        return unpadded_data
```

Bagian ini mengimplementasikan mode Cipher Feedback dari cipher blok.

3.4.4 Output Feedback (OFB)

```python
class OFBMode(CipherMode):
    def __init__(self, cipher : NewBlockCipher, iv):
        super().__init__(cipher)
        if len(iv) != self.cipher.block_size:
            raise ValueError(f"IV harus berukuran {self.cipher.block_size} bytes.")
        self.iv = iv

    def encrypt(self, plaintext):
        padded_plaintext = PKCS7Padding.pad(plaintext, self.cipher.block_size)
        ciphertext = b''
        feedback = self.iv

        for i in range(0, len(padded_plaintext), self.cipher.block_size):
            # Enkripsi feedback
            encrypted_feedback = self.cipher.encrypt_block_bytes(feedback)
            block = padded_plaintext[i:i + self.cipher.block_size]
            # XOR dengan blok plaintext
            encrypted_block = bytes(a ^ b for a, b in zip(block, encrypted_feedback))
            ciphertext += encrypted_block
            feedback = encrypted_feedback  # Update feedback dengan hasil enkripsi feedback

        return ciphertext

    def decrypt(self, ciphertext):
        if len(ciphertext) % self.cipher.block_size != 0:
            raise ValueError("Panjang ciphertext harus kelipatan dari ukuran blok.")

        decrypted_data = b''
        feedback = self.iv

        for i in range(0, len(ciphertext), self.cipher.block_size):
            # Enkripsi feedback
            encrypted_feedback = self.cipher.encrypt_block_bytes(feedback)
            block = ciphertext[i:i + self.cipher.block_size]
            # XOR dengan blok ciphertext
            original_block = bytes(a ^ b for a, b in zip(block, encrypted_feedback))
            decrypted_data += original_block
            feedback = encrypted_feedback  # Update feedback dengan hasil enkripsi feedback

        unpadded_data = PKCS7Padding.unpad(decrypted_data)
        return unpadded_data
```

Bagian ini mengimplementasikan mode Output Feedback dari cipher blok.

3.4.5 Counter (CTR)

```python
class CTRMode(CipherMode):
    def __init__(self, cipher : NewBlockCipher, nonce):
        super().__init__(cipher)
        if len(nonce) != self.cipher.block_size // 2:
            raise ValueError(f"Nonce harus berukuran {self.cipher.block_size // 2} bytes.")
        self.nonce = nonce
        self.counter = 0

    def get_counter_block(self):
        # Membuat blok counter dengan menggabungkan nonce dan counter
        counter_bytes = self.counter.to_bytes(self.cipher.block_size // 2, 'big')
        return self.nonce + counter_bytes

    def encrypt(self, plaintext):
        padded_plaintext = PKCS7Padding.pad(plaintext, self.cipher.block_size)
        ciphertext = b''

        for i in range(0, len(padded_plaintext), self.cipher.block_size):
            block = padded_plaintext[i:i + self.cipher.block_size]
            counter_block = self.get_counter_block()
            encrypted_counter = self.cipher.encrypt_block_bytes(counter_block)
            encrypted_block = bytes(a ^ b for a, b in zip(block, encrypted_counter))
            ciphertext += encrypted_block
            self.counter += 1  # Increment counter untuk blok berikutnya

        self.counter = 0  # Reset counter setelah enkripsi selesai

        return ciphertext

    def decrypt(self, ciphertext):
        if len(ciphertext) % self.cipher.block_size != 0:
            raise ValueError("Panjang ciphertext harus kelipatan dari ukuran blok.")

        decrypted_data = b''

        for i in range(0, len(ciphertext), self.cipher.block_size):
            block = ciphertext[i:i + self.cipher.block_size]
            counter_block = self.get_counter_block()
            encrypted_counter = self.cipher.encrypt_block_bytes(counter_block)
            original_block = bytes(a ^ b for a, b in zip(block, encrypted_counter))
            decrypted_data += original_block
            self.counter += 1  # Increment counter untuk blok berikutnya

        self.counter = 0  # Reset counter setelah dekripsi selesai

        unpadded_data = PKCS7Padding.unpad(decrypted_data)
        return unpadded_data
```

Bagian ini mengimplementasikan mode Counter dari cipher blok.

3.5 Metode *PKCS#7 Padding*

```python
class PKCS7Padding:
    @staticmethod
    def pad(value, block_size):
        # Memasukkan padding ke dalam input value agar panjangnya menjadi kelipatan dari block_size.
        # Padding scheme yang digunakan adalah PKCS#7.
        padding_length = block_size - (len(value) % block_size)
        padding = bytes([padding_length] * padding_length)
        return value + padding

    @staticmethod
    def unpad(value):
        # Menghapus padding dari input value yang telah dipad.
        # Padding scheme yang digunakan adalah PKCS#7.
        if not value:
            raise ValueError("Nilai input tidak boleh kosong.")

        padding_length = value[-1]

        if padding_length < 1 or padding_length > len(value):
            raise ValueError("Panjang padding tidak valid.")

        if value[-padding_length:] != bytes([padding_length] * padding_length):
            raise ValueError("Bytes padding tidak valid.")

        return value[:-padding_length]
```

Bagian ini mengimplementasikan padding agar panjang bit sama dengan kelipatan dari BLOCK_SIZE 8 byte (64 bit). Selain padding, ada juga penghapusan padding dari blok bit yang sudah ada padding.

BAB IV
IMPLEMENTASI DAN SIMULASI

4.1 Implementasi Aplikasi

```python
from cipher.algorithm import NewBlockCipher
from cipher.modes import ECBMode, CBCMode, CFBMode, OFBMode, CTRMode
from cipher.file_cipher import FileCipher

if __name__ == "__main__":
    # Penggunaan ECBMode dengan algoritma NewBlockCipher
    print("=== New Block Cipher ===")
    print("Aplikasi block cipher sederhana dengan berbagai mode operasi.\n\n")
    master_key = input("Masukkan kunci cipher (8 karakter): ").encode('utf-8')
    cipher = NewBlockCipher(master_key)
    mode_option = input("""
        Pilih mode operasi:
        1. ECB (Electronic Codebook)
        2. CBC (Cipher Block Chaining)
        3. CFB (Cipher Feedback)
        4. OFB (Output Feedback)
        5. CTR (Counter)
        6. Exit
        """)
    match mode_option:
        case "1":
                cipher_mode = ECBMode(cipher)
        case "2":
                iv = input("Masukkan initial vector (string 8 karakter): ").encode('utf-8')
                cipher_mode = CBCMode(cipher, iv)
        case "3":
                iv = input("Masukkan initial vector (string 8 karakter): ").encode('utf-8')
                cipher_mode = CFBMode(cipher, iv)
        case "4":
                iv = input("Masukkan initial vector (string 8 karakter): ").encode('utf-8')
                cipher_mode = OFBMode(cipher, iv)
        case "5":
                nonce = input("Masukkan nonce (string 4 karakter): ").encode('utf-8')
                cipher_mode = CTRMode(cipher, nonce)
        case "6":
                print("Keluar dari program.")
                exit()
        case _:
                print("Pilihan tidak valid. Silakan pilih 1, 2, 3, 4, 5, atau 6.")
                exit()

```

```python
    while True:
        menu = """
        Pilih mode operasi (ECB):
        1. Enkripsi teks
        2. Dekripsi teks
        3. Enkripsi file
        4. Dekripsi file
        5. Keluar
        """
        print(menu)
        option = input("Masukkan pilihan (1/2/3): ")
        if option == "1":
                plaintext = input("Masukkan plaintext (utf-8): ").encode('utf-8')
                ciphertext = cipher_mode.encrypt(plaintext)
                ciphertext_hex = ciphertext.hex()
                print("Ciphertext (hex):", ciphertext_hex)
        elif option == "2":
                ciphertext_hex = input("Masukkan ciphertext (hex): ")
                ciphertext = bytes.fromhex(ciphertext_hex)
                decrypted_text = cipher_mode.decrypt(ciphertext)
                print("Decrypted text:", decrypted_text.decode('utf-8'))
        elif option == "3":
                input_file_name = input("Masukkan nama file di folder files (contoh: plaintext.txt): ")
                input_file_path = "files/" + input_file_name

                output_file_name = input_file_name.split('.')[0] + "_encrypted." + input_file_name.split('.')[1]
                output_file_path = "files/" + output_file_name

                FileCipher.encrypt_file(cipher_mode, input_file_path, output_file_path)
                print(f"File terenkripsi disimpan di: {output_file_path}")
        elif option == "4":
                input_file_name = input("Masukkan nama file terenkripsi di folder files (contoh: ciphertext.txt): ")
                input_file_path = "files/" + input_file_name

                output_file_name = input_file_name.split('_')[0] + "_decrypted." + input_file_name.split('.')[1]
                output_file_path = "files/" + output_file_name

                FileCipher.decrypt_file(cipher_mode, input_file_path, output_file_path)
                print(f"File terdekripsi disimpan di: {output_file_path}")
        elif option == "5":
                print("Keluar dari program.")
                break
        else:
                print("Pilihan tidak valid. Silakan pilih 1, 2, atau 3.")
```

Aplikasi ini dijalankan di *Python runtime* (CLI) dengan mengetik *command* py app.py di terminal. Pada bagian ini, aplikasi utama meminta input *master key* (string 8 karakter) kemudian menyediakan pilihan mode cipher (ECB, CBC, CFB, OFB, CTR). Jika CBC, CFB, atau OFB dipilih, aplikasi akan meminta input berupa *initial vector* (string 8 karakter). Jika CTR dipilih, aplikasi akan meminta input berupa *nonce* (string 4 karakter). Setelah salah satu mode dipilih, aplikasi masuk ke bagian utama.

Pada bagian utama, terdapat 5 opsi: enkripsi teks, dekripsi teks, enkripsi file, dekripsi teks, dan keluar. Pada bagian enkripsi teks, aplikasi akan menanyakan string yang akan dienkripsi (UTF-8). Cipherteks hasil enkripsi lalu ditampilkan dalam bentuk heksadesimal. Pada bagian dekripsi teks, aplikasi akan menanyakan string heksadesimal yang akan didekripsi. Setelah dekripsi, teks ditampilkan dalam format UTF-8. Sementara itu, pada bagian enkripsi dan dekripsi

file, aplikasi akan mengenkripsi atau mendekripsi berkas (*file*) berdasarkan *path* yang dimasukkan pengguna aplikasi.

4.2 Simulasi

No.	Kunci	Mode	Plainteks	Cipherteks (hex)	Teks terdekripsi
1	12345678	ECB	aku cinta indonesia	ee0bc51104311845bdf70f195cf1ec6aa31d9fe9ee73fd42	aku cinta indonesia
2	12345678	ECB	Aku cinta Indonesia.	b25da2b8ee27aa0d0fc81a85c762ae245f2bc1d6794d20ea	Aku cinta Indonesia.
3	87654321	ECB	Aku cinta Indonesia.	4f0e2c8c122da3cd8608fb334c94173fdc9b32b9abfa4009	Aku cinta Indonesia.

BAB V
ANALISIS KEAMANAN

5.1 Avalanche Effect

Avalanche Effect merupakan fenomena dalam kriptografi di mana perubahan kecil pada input (plainteks), misalnya satu bit, menghasilkan perubahan besar pada output (cipherteks). Efek ini dapat diukur untuk memastikan keamanan algoritma block cipher. Semakin besar persentase perubahan bit antara cipherteks asli dan cipherteks yang dimodifikasi, semakin aman algoritma block cipher tersebut terhadap serangan analisis diferensial.

No.	Plainteks A	Plainteks B	Cipherteks A (hex)	Cipherteks B (hex)	Persentase
1	aku cinta indonesia	Aku Cinta Indonesia.	ee0bc51104311845 bdf70f195cf1ec6aa3 1d9fe9ee73fd42	b25da2b8ee27aa0d0 fc81a85c762ae245f2 bc1d6794d20ea	53.65%

5.2 Entropi

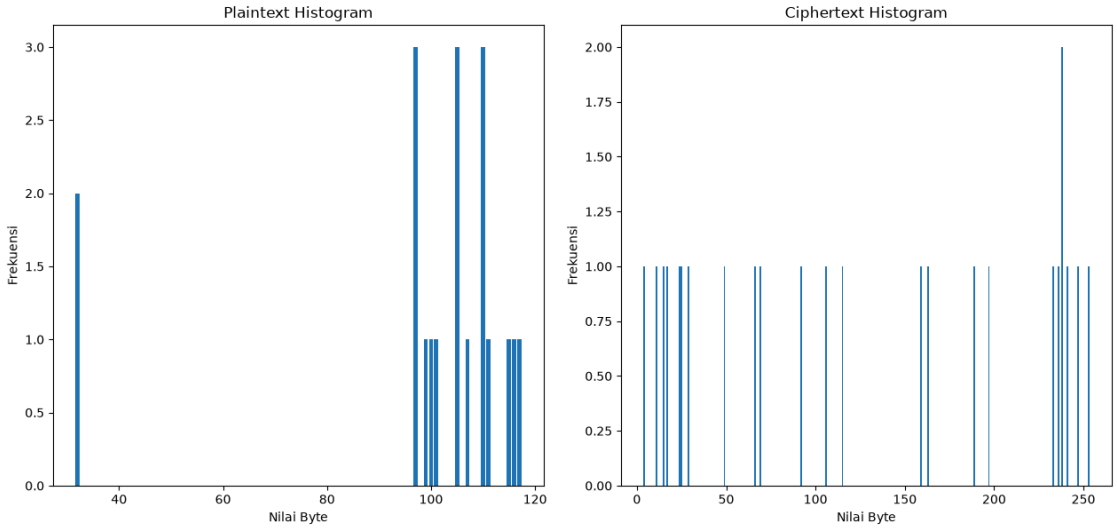
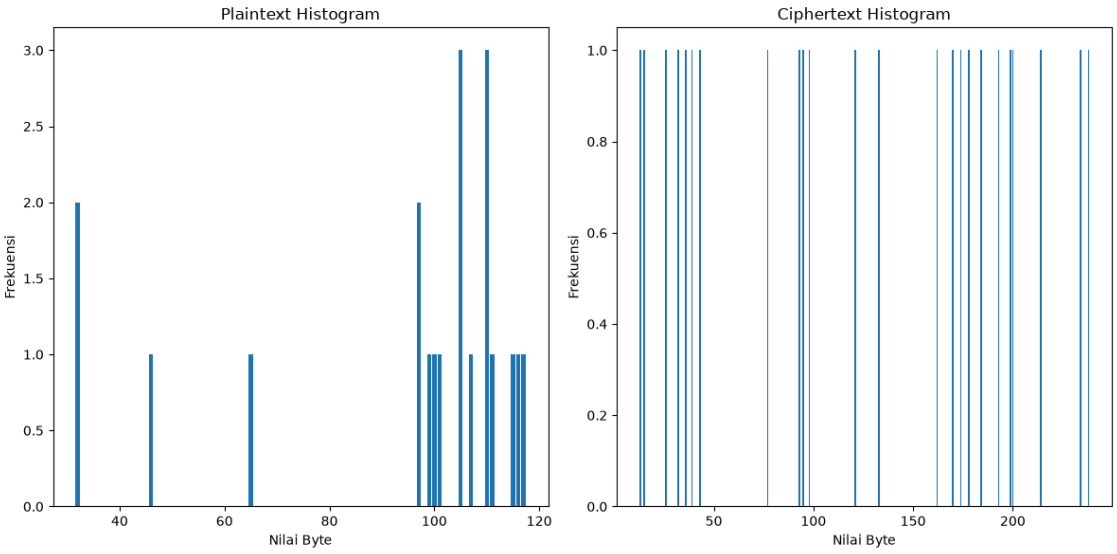
Entropi merupakan ukuran keacakan dalam suatu sistem. Dalam kriptografi, entropi digunakan untuk mengukur derajat keacakan dari cipherteks yang dihasilkan oleh algoritma block cipher, Semakin tinggi entropi, semakin sulit penyerang untuk memprediksi atau menebak plainteks asli dari cipherteks.

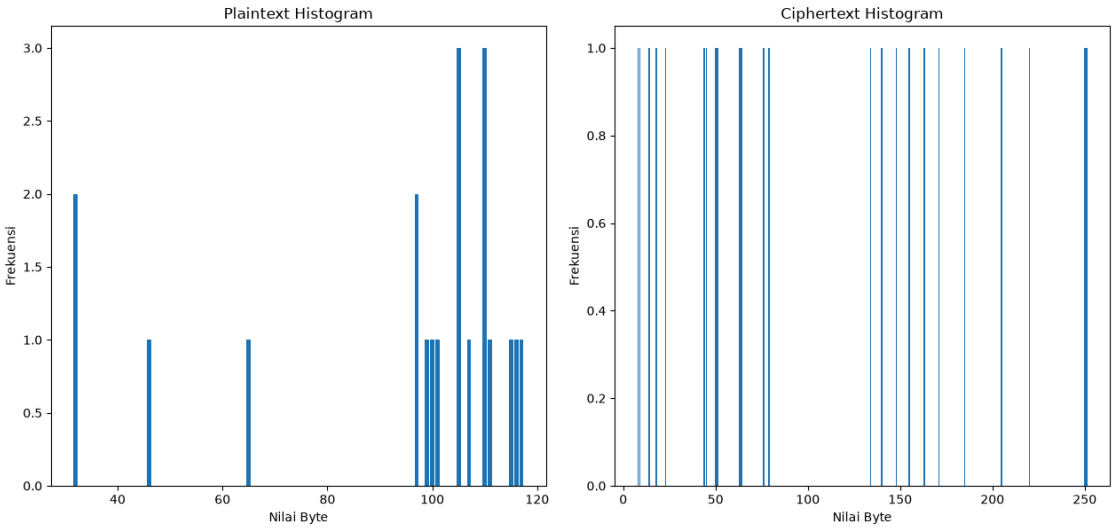
No.	Cipherteks	Entropi
1	ee0bc51104311845bdf70f195cf1ec6aa31d9fe9ee73fd42	36.01 bit/byte
2	b25da2b8ee27aa0d0fc81a85c762ae245f2bc1d6794d20ea	36.68 bit/byte
3	4f0e2c8c122da3cd8608fb334c94173fdc9b32b9abfa4009	36.68 bit/byte

5.3 Distribusi Histogram

Distribusi histogram merupakan representasi visual dari frekuensi kemunculan setiap nilai byte dalam data. Dalam kriptografi, distribusi histogram digunakan untuk menganalisis pola dan karakteristik dari plainteks dan cipherteks yang dihasilkan oleh algoritma block cipher. Dengan membandingkan histogram antara plainteks dan cipherteks, dapat dievaluasi sejauh mana algoritma mengacak data dan menghilangkan pola yang mungkin ada dalam plainteks. Distribusi histogram dari cipherteks yang baik harus seragam dan tidak menunjukkan pola yang jelas supaya efektif dalam mengacak data dan melindungi informasi dari analisis statistik.

No.	Distribusi Histogram	
1	Plainteks	Cipherteks (hex)

	aku cinta indonesia	ee0bc51104311845bdf70f195cf1ec6aa31d9fe9ee73fd42
		
2	Plainteks	Cipherteks (hex)
	Aku cinta Indonesia.	b25da2b8ee27aa0d0fc81a85c762ae245f2bc1d6794d20ea
		
3	Plainteks	Cipherteks (hex)
	Aku cinta Indonesia.	4f0e2c8c122da3cd8608fb334c94173fdc9b32b9abfa4009


Plaintext Histogram
Frekuensi
Nilai Byte
0.0
0.5
1.0
1.5
2.0
2.5
3.0
40
60
80
100
120
Ciphertext Histogram
Frekuensi
Nilai Byte
0.0
0.2
0.4
0.6
0.8
1.0
0
50
100
150
200
250

BAB VI
KESIMPULAN DAN SARAN

6.1 Kesimpulan

Berdasarkan hasil simulasi dan analisis keamanan, algoritma *block cipher* "baru" ini berhasil menunjukkan sifat *confusion* dan *diffusion* pada beberapa cipherteks dengan variasi kunci dan plainteks.

6.2 Saran

Sebaiknya ditambahkan fungsi *MixColumns* seperti AES menggunakan Medan Galois supaya algoritma tersebut semakin sulit untuk dipecahkan.

DAFTAR PUSTAKA

GeeksforGeeks. “Block Cipher modes of Operation.” *GeeksforGeeks*, 2026, https://www.geeksforgeeks.org/ethical-hacking/block-cipher-modes-of-operation/. Accessed 13 October 2026.

Munir, Rinaldi. “08 - Block Cipher (Bagian 1).” *Homepage Rinaldi Munir*, 2026, https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/08-Block-Cipher-Bagian1-(2026).pdf. Accessed 13 10 2026.

NIST. “Advanced Encryption Standard (AES).” *NIST Technical Series Publications*, 2001, https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.197-upd1.pdf. Accessed 13 October 2026.

LAMPIRAN

Link repository: https://github.com/RealNath/new-block-cipher

Nama	NIM	Pembagian Tugas
Jonathan Levi	13523132	● Menentukan arsitektur model, nilai S-box, P-box ● Mengimplementasikan fungsi apply S-box, apply P-box, enkripsi berdasarkan jaringan Feistel ● Mengimplementasikan teknik operasi tambahan rotasi bit ● Memastikan memenuhi prinsip confusion dan diffusion ● Membantu menulis laporan.
Jovandra Otniel P. S.	13523141	● Mengimplementasikan mode-mode cipher ECB, CBC, CFB, OFB, dan CTR. ● Mengimplementasikan fungsi pembangkitan kunci ronde (kunci urutan). ● Mengimplementasikan kelas PKCS7Padding untuk mengimplementasikan metode *padding* berbasis PKCS#7. ● Mengimplementasikan aplikasi utama (app.py) yang dapat dijalankan di terminal. ● Membantu menulis laporan.
Anas Ghazi Al Gifari	13523159	● Mengimplementasikan Avalance Effect ● Mengimplementasikan Entropi ● Mengimplementasikan Distribusi Histogram ● Membantu menulis laporan